

Akash Yadav

DevOps & Cloud Engineer | Kubernetes | AWS | Fintech Infrastructure

ARCHITECTURE DEEP-DIVE

FinTech Payments Platform Architecture

An architecture review of a production FinTech Payments Platform on AWS EKS — the services, the reasoning behind every major decision, the tradeoffs accepted, and how the system behaves when things fail.

AWS EKS · Istio · Kafka (MSK) · ArgoCD · OPA · Falco · OpenTelemetry · Terraform

What This Platform Actually Does

A note on what this document is

This is a reference architecture inspired by production cloud-native patterns and real-world fintech engineering practices, written for educational and portfolio purposes. It illustrates how these decisions fit together in a payments platform; it does not claim ownership of, or direct operational experience running, any specific named company's production system beyond what is explicitly described here.

This is a live FinTech Payments Platform — real payments, real user data, real regulators looking over its shoulder. Nothing here started on a whiteboard as a clean diagram; it took shape under compliance audits, peak transaction-volume spikes, and a hard rule that we could never take the system down to ship a fix.

This is a FinTech Payments Platform first: the primary focus of this document is payment processing and the cloud-native infrastructure that supports it. The platform is built around one core capability — moving money safely and proving, to a regulator's satisfaction, that it happened correctly. Payment processing, fraud detection, AML, KYC, ledger, and compliance reporting form the spine of the system — every other service is designed to plug into the event stream they produce, not the reverse. A supporting set of business services — customer notifications, reporting and analytics, and customer loyalty — also consume the same payment events, but they're deliberately kept outside the trust boundary that payment and compliance code live in: a slow notification provider or a buggy loyalty deploy is never allowed to put a payment at risk.

The problem underneath all of it is simple to state and hard to solve: one person clicking “pay” sets off several things on the critical path, none of which are allowed to block each other, and none of which are allowed to silently fail. Fraud has to check the transaction. A receipt has to go out. The ledger has to record the money moving. Compliance needs a clean audit trail it can hand to a regulator without notice. Downstream business consumers — loyalty accrual among them — pick up the same event too, but on their own failure terms: a slow or broken loyalty deploy delays a rewards credit, never a payment. If any one of the critical-path steps turns into a synchronous dependency, the whole payment flow inherits its latency and its failure modes.

Everything below — the services, the namespaces, the event backbone, and the failure handling — is the answer we landed on after trying, and breaking, a more naively coupled version first. Where relevant, this document explains why an alternative was rejected, not just what was built.

66+ pods running across the cluster	7 namespaces, one per domain boundary	3 node groups tuned for different workloads	1:1 service-to-database ratio — no shared schemas
---	---	---	---

These numbers describe the shape of the running production cluster, not a target architecture. Node count runs higher relative to pod count than a tightly bin-packed cluster would suggest — headroom is kept deliberately for Spot reclamation, canary rollouts, and node-group-specific instance sizing (see Production Metrics), rather than optimizing for maximum pods-per-node.

Database Ownership

Every microservice owns its own datastore, and no service reaches into another service's database directly. Payment, Fraud, Auth, Ledger, and Loyalty each expose their data only through an API or a Kafka event — never a shared schema, never a shared connection pool. This buys three things: independent deployability (a schema migration in Loyalty can never lock a table Payment depends on), fault isolation (a slow query in Reporting cannot starve Ledger's connection pool), and a much smaller blast radius when something goes wrong.

The tradeoff is real, and worth naming rather than hiding: distributed data means eventual consistency across service boundaries, no cheap cross-service JOINS, and more moving parts to keep those boundaries in sync. We accept that

cost and manage it with event-driven integration — the Outbox Pattern for reliable publishing, idempotent consumers on the read side, and purpose-built read models (like Reporting's replica-backed aggregates) instead of ad-hoc cross-service queries.

This is not presented as a universal rule. A five-person startup is often better served by one shared database — the coordination cost of database-per-service isn't worth paying until the org and the traffic justify it. A modular monolith with clean internal boundaries can defer this decision entirely, and most companies still run analytics off a shared warehouse fed by CDC from many services rather than forcing every service to expose its own reporting API. For this platform, at this scale, with these compliance boundaries, database-per-service was the right call — full service autonomy and fault isolation were worth more than the convenience of a shared schema.

How the Cluster Is Actually Carved Up

A single EKS cluster hosts everything, but it isn't one flat pool of pods. It's split into seven namespaces, and the split isn't cosmetic — each namespace is its own network-isolated blast radius with its own NetworkPolicy rules, so a misbehaving pod in one domain physically cannot reach a database or service in another domain without going through a defined API.

Namespace	What lives here	Why it's separated out
istio-system	Istio ingress gateway, mesh control plane, sidecar config	Mesh plumbing needs to be upgraded and scaled on its own release cycle, independent of application code.
payment-services	Payment Gateway, Transaction Processor, Ledger	The narrowest practical scope for anything that touches card data or money movement inside the cluster — tighter access controls and a smaller operational blast radius. Namespace isolation alone is not treated as a PCI-DSS compliance boundary here; see Architecture Tradeoffs and Known Limitations & Operational Edge Cases for what a harder compliance boundary would additionally require.
risk-and-compliance	Fraud Detection, AML, Reporting	Different regulatory owners and retention rules than the rest of the platform; audit access is scoped to this namespace alone.
user-services	Auth, Authorization (OPA), KYC Verification, User Profile	Identity is a dependency of almost everything else, so it needs to scale and fail independently of business logic.
other-services	Notifications, Loyalty Engine, and 20+ other business-logic microservices	Fast-moving, marketing-driven logic that redeploys far more often than payment code — isolating it keeps risky deploys away from the money path.
monitoring	Prometheus, Alertmanager, Grafana, Jaeger, OTel Collector	Observability has to keep running even if an application namespace is unhealthy, so it's kept structurally separate.
security	Falco, OPA policy bundles, Secrets Store CSI Driver	Runtime security and secrets access shouldn't live in the same namespace as the workloads it's watching.

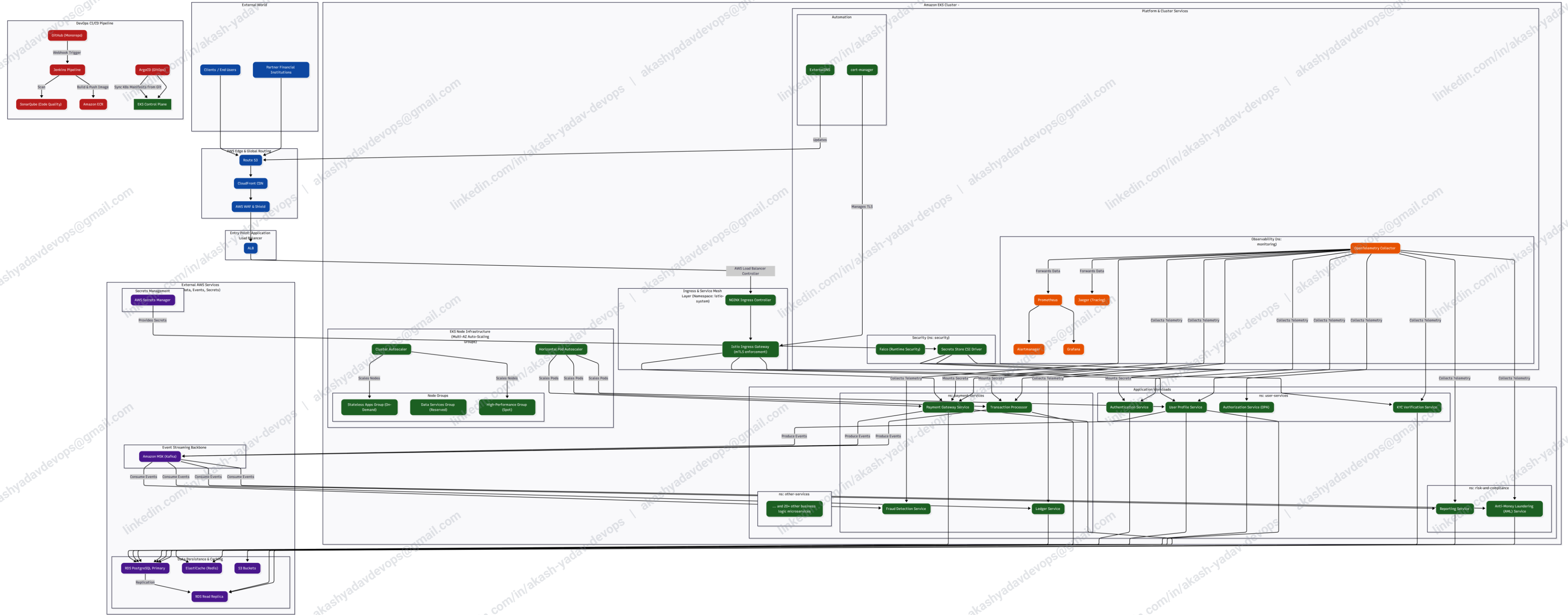
Cross-namespace calls only happen through defined service APIs — never a direct database connection, never a shared schema.

A question worth asking directly: why one EKS cluster with seven namespaces instead of seven separate clusters? Separate clusters give harder isolation — a control-plane incident in one can't touch another — but they multiply the operational surface: seven sets of upgrades, seven sets of IAM wiring, seven service meshes to keep in sync, and cross-domain traffic that now has to cross cluster boundaries instead of staying inside one mesh. At this service count, namespace-level isolation plus NetworkPolicy plus Istio's per-namespace mTLS gets most of the blast-radius reduction of separate clusters at a fraction of the operational cost. If a single domain's compliance requirements ever demanded a hard control-plane boundary — a common trigger is a regulator requiring physically separate infrastructure — that namespace would be the first candidate to split into its own cluster.

Three EC2 node groups sit underneath these namespaces, each tuned for a different kind of workload rather than one generic node type trying to serve everyone:

Node group	Type	What runs here	Why
Stateless Apps	On-Demand	APIs, gateways, frontends	These need to be available the instant traffic arrives — no interruption risk, no waiting on Spot capacity.
Data Services	Reserved	Kafka brokers, Redis, caching layers	Always-on by nature, so reserved pricing meaningfully cuts cost compared to on-demand for the same footprint (illustrative — exact savings depend on commitment term and instance mix, not a measured figure).
High-Performance	Spot	Batch jobs, ML scoring, reporting	Fault-tolerant and non-critical — if a Spot node gets reclaimed mid-job, the job just retries. Cheapest way to run this work.

System Architecture — Full Diagram



Full system architecture — every layer, namespace, event backbone, and external dependency in one diagram. See Kafka in Depth, the Request Path, and What Happens the Moment Someone Pays for focused views of the event flow, network/security boundaries, and payment request path respectively. Designed by Akash Yadav.

The Services, One at a Time

This is the part most architecture write-ups skip: not just naming the boxes in the diagram, but explaining why each one is a separate service instead of a function call inside something bigger, who it actually talks to, and what breaks if it goes away. Every service owns its own datastore where it needs one.

payment-services

Payment Gateway · *payment-services*

Front door, smallest possible footprint. Payment Gateway validates the charge request and forwards card data directly to the external payment processor for authorization and tokenization, rather than tokenizing or storing it in-house — the common production pattern for keeping card data out of a platform's own PCI scope. It wasn't merged with Transaction Processor because doing so would pull card-data-handling review over a much larger, faster-changing codebase; every feature added to transaction logic would then need to clear that review for a boundary only one of the two services actually needs.

Where it sits operationally. This is the one hop in the platform where synchronous request/response is the right call, not an anti-pattern — the customer is actively waiting. It scales on CPU plus a custom metric on in-flight requests to the processor, since the external network hop is usually the bottleneck, not CPU, and it holds no persistent store of its own beyond short-lived idempotency keys in Redis that stop a retried request from double-charging a card. P95 target is under 300ms for the internal hop; the external processor call typically dominates total latency and is budgeted separately. If this service disappeared, payments stop completely — a hard dependency, which is why it runs on the On-Demand node group behind an aggressive HPA and never on Spot capacity.

Transaction Processor · *payment-services*

The system of record. Transaction Processor takes a confirmed charge, writes the authoritative transaction row, and publishes the event everything downstream reacts to. Business transaction logic is kept separate from the mechanics of talking to a payment processor so the two can change on their own schedules. It wasn't folded into Ledger either — that would mix a service that has to react quickly to a single payment event with one that has to guarantee immutable, auditable history, two different operational profiles with two different teams' concerns.

Where it sits operationally. It writes synchronously to its own Postgres schema on RDS — never shared with Ledger, even though the two are tightly related — then publishes PaymentCompleted to Kafka asynchronously for every downstream consumer. Database writes are the limiting factor as volume scales, so PgBouncer connection pooling matters more here than raw pod count. P95 target is under 150ms for the DB write plus outbox insert; the Kafka publish itself is decoupled from the customer-facing response. If it disappeared, payments would still be charged by the processor, but nothing downstream would know — no fraud check, no receipt, no ledger entry — which is exactly why the write and the publish are wrapped in the Outbox Pattern rather than treated as two independent steps.

Ledger · *payment-services*

The append-only source of truth. Ledger is double-entry bookkeeping for every cent that moves through the platform. It has to be immutable and independently auditable, so no other service, however trusted, writes to its schema. A single writer path — consuming Transaction Processor's events — keeps a reconciliation investigation tractable; the moment more than one service could plausibly write to a shared ledger table, every one of them becomes a candidate suspect when something doesn't add up.

Where it sits operationally. It consumes transaction events asynchronously and is never called synchronously by anything outside payment-services. Write throughput is intentionally modest — append-only, sequential inserts — while read-heavy scaling is handled by a dedicated read replica. Because it isn't customer-facing, its latency budget is generous, seconds rather than milliseconds; correctness matters far more than speed here. If it disappeared, money would still move,

but there would be no source of truth to reconcile against — audits and financial reporting stop being possible within minutes of an incident.

risk-and-compliance

Fraud Detection · risk-and-compliance

Scores every transaction, blocks nothing by default. Fraud Detection consumes the `PaymentCompleted` event, runs it through a rule engine plus an ML scoring model, and independently flags or clears the transaction. The ML pass in particular can take longer than a customer should ever wait for a payment to clear, so it runs off the event stream instead of sitting in the payment path. It ships new rules and model versions on a weekly cadence — a different rhythm than payment processing — so keeping it separate means those deploys never risk payment availability.

Where it sits operationally. It's a purely asynchronous, independent Kafka consumer group off `PaymentCompleted`, emitting `FraudFlagged` when it finds something, and it scales on consumer lag rather than CPU. It owns its own store for rule sets and model features, with Redis caching recent feature lookups. There's no hard SLA against the payment flow — internally it targets scoring within seconds of publish. If it disappeared, transactions still complete: this service fails open by design, since blocking legitimate payments is worse than temporarily losing fraud coverage. Unsourced transactions queue in Kafka and are caught up once the service recovers, with an alert firing on the backlog immediately.

AML (Anti-Money-Laundering) · risk-and-compliance

Same event, different regulator. AML consumes the same `PaymentCompleted` event as Fraud Detection, checks it against sanctions lists and compliance rules, and files a Suspicious Activity Report when required. It answers to a different regulator than Fraud, with different retention rules and a different audit trail, so bundling the two would mean one team's compliance obligations leaking into another's service — and every change would need to satisfy both regulatory regimes at once.

Where it sits operationally. It reads off its own independent Kafka consumer group on the same topic Fraud reads from; the two never call each other. Volume is steady rather than bursty, so a small fixed pod count with headroom is usually enough. It keeps its own compliance datastore plus S3 for long-retention SAR filings and evidence exports. Regulatory turnaround windows are measured in hours to days, not seconds — the one service on this platform where near-term latency isn't the design driver. If it disappeared, compliance obligations start accumulating unmet; that's a slow-burn failure rather than a customer-visible one, but it's treated as high-severity because regulatory exposure compounds daily.

Reporting · risk-and-compliance

Deliberately off the hot path. Reporting aggregates data out of Ledger, Fraud, and AML to produce compliance reports and internal dashboards. It's a read-heavy, bursty workload that would compete for resources exactly when the platform is busiest if it ran against the live payment path, so it's split out and tuned — and scaled — independently from the low-latency transactional side of the system.

Where it sits operationally. It reads from dedicated replicas — never the primary — of Ledger, Fraud, and AML, and writes nothing back into any of them. It's scheduled onto Spot nodes and scaled by job concurrency rather than a standing HPA, since most of its work runs in batch windows. Reports are typically minutes to hours behind live data, an accepted tradeoff for staying off the hot path, and there's no customer impact if it disappears — internal dashboards and scheduled compliance exports simply stop updating until it's restored.

user-services

Auth Service · user-services

Almost everything depends on this. Auth Service authenticates users and issues the JWTs that every other service trusts. Nearly everything downstream depends on it succeeding fast, so it needs to scale on its own during login bursts without waiting on unrelated services. It's kept separate from general user-profile logic because authentication has a fundamentally different threat model and change cadence, and isolating it keeps the credential-handling surface as small as possible.

Where it sits operationally. Every authenticated request needs a synchronous, fast answer, so Auth runs an aggressive HPA tuned for login bursts — morning traffic, marketing pushes — backed by Redis for session state so pods stay stateless and interchangeable. User credentials live in its own RDS schema. P95 target is under 100ms, the single internal hop most sensitive to added latency since it sits in front of nearly everything else. If it disappeared, nobody can log in or refresh a session — a full outage for most of the platform, which is why it carries the tightest SLOs on the whole system.

Authorization (OPA) · user-services

A different question from authentication. Authorization answers “can this user do this specific action on this specific resource” — a genuinely different problem from “who is this user.” Running it as Rego policy through OPA means access rules can change and roll out without touching or redeploying application code; hard-coding those checks into each service instead would turn every policy change into N deploys rather than one policy push, which alone justifies the separation.

Where it sits operationally. It runs as a lightweight sidecar-adjacent service, queried synchronously at the API boundary by both Istio and application services, and owns nothing but policy bundles — no user data of its own. Policy evaluation is fast enough that scaling is rarely the bottleneck compared to the services calling it, and the target is sub-10ms since it sits inline on the request path for protected resources. If it disappeared, it fails closed: requests requiring a policy decision are denied by default rather than silently allowed, trading availability for safety on purpose.

KYC Verification · user-services

Bounded by a vendor's SLA, not this platform's. KYC Verification handles identity verification during onboarding — document capture, liveness checks, and calls out to a third-party identity vendor. That work is dominated by slow external calls that can take minutes, which would drag down Auth's sub-100ms budget if the two were bundled together; the vendor dependency and multi-minute turnaround are fundamentally incompatible with how fast Auth has to stay. Operationally, it runs asynchronously — kicking off a verification workflow and writing status back once the vendor responds — scaling on queue depth of pending verifications rather than request rate, with its own schema for verification status and document metadata and raw documents in S3. If it disappeared, new user onboarding stalls, but existing verified users are unaffected since their status is already persisted.

other-services

Notification Service · other-services

Best-effort by design, never in the critical path. Notification Service consumes PaymentCompleted and sends SMS, email, and push confirmations in parallel through third-party providers. Delivery through an external gateway is inherently best-effort and sometimes slow, and a provider outage or rate limit should never be able to slow down or fail a payment — full separation from the payment path guarantees that.

Where it sits operationally. It's an asynchronous Kafka consumer with its own retry logic and dead-letter handling for provider failures, scaling on consumer lag — though provider rate limits are usually the real ceiling, so scaling pods beyond that point doesn't help. It keeps only a delivery status log, no financial data. Delivery is best-effort, typically seconds to a couple of minutes depending on the provider. If it disappeared, payments still succeed; customers simply don't get a receipt until the service recovers and works through its backlog.

Loyalty Engine · other-services

Marketing's cadence, not engineering's. Loyalty Engine credits points, updates tier status, and runs promotional logic off the same payment event. Loyalty rules change constantly, driven by marketing rather than engineering cadence, and isolating this service means a risky loyalty deploy can never take down or slow down anything on the money path — the deploy-frequency mismatch alone justifies the split.

Where it sits operationally. It's a standard asynchronous Kafka consumer off PaymentCompleted, scaled by a normal HPA on CPU since traffic tracks payment volume, with its own dedicated Postgres schema for points and tier history. There's no customer-facing SLA — points typically reflect within a minute of payment. If it disappeared, payments and receipts are unaffected; points simply stop accruing until the service is back, and a backfill job reconciles the gap.

Activity Tracker (Audit) · other-services

Neutral by design. Activity Tracker is a read-only observer that consumes events across the platform and writes an immutable audit record for every one of them. The audit requirement cuts across every domain, so rather than have each team own a slice of the compliance surface inside their own service, it's centralized here as a pure observer — it depends on nothing, and nothing depends on it. It consumes broadly across Kafka topics, never publishes anything anyone else depends on, and scales on aggregate event volume; its only data store is S3, versioned and access-logged for SOC 2 evidence. Durability and completeness matter more than speed here, and if it disappeared there's no functional impact on the platform — just a growing audit gap that's treated as a compliance-severity incident even though nothing customer-facing breaks.

How the Services Actually Talk to Each Other

Early versions of this platform chained these steps together synchronously — Payment Gateway called Fraud, which called Notifications, which called Loyalty, all as blocking API calls. It worked until any one of those had a bad day, at which point a slow fraud model or a flaky SMS provider could stall or fail a payment that had, in reality, already gone through fine.

The fix was to stop treating this as a call chain and start treating it as one event with many independent listeners. The Transaction Processor publishes a single PaymentCompleted event to Amazon MSK, and Fraud Detection, AML, Notifications, and the Loyalty Engine each consume it on their own consumer group, at their own pace, in parallel. None of them can block the others, and none of them can block the payment flow that already completed by the time they start working.

Event	Published by	Consumed by	Partitions / Retention
PaymentCompleted	Transaction Processor	Fraud, AML, Notifications, Loyalty, Audit	24 partitions, key = account ID, 7-day retention
FraudFlagged	Fraud Detection	Ledger (hold/reverse), Notifications, Audit	12 partitions, key = transaction ID, 30-day retention
SARFiled	AML	Reporting, Audit	6 partitions, key = case ID, 7-year retention (compliance)

Moving from synchronous calls to this event-driven model removed the head-of-line blocking synchronous chains create, and made partial failures invisible to the end user — if the Notification Service is down, a customer's payment still succeeds and the receipt just arrives a little late once the service recovers, instead of the whole payment failing. (This is a qualitative comparison rather than a benchmarked figure; see Production Metrics for the platform's illustrative latency numbers.)

Architecture Tradeoffs

None of the decisions below were free. Each one traded something away on purpose, and each one was chosen after an alternative was seriously considered and rejected for a specific, statable reason — not because it was fashionable.

Event streaming (Kafka) instead of synchronous REST

The problem, and what else was on the table. A single payment fans out to six downstream processes. Chaining them as synchronous API calls means one slow or failing service degrades the entire payment flow — the option rejected first. A workflow-engine orchestrator was considered too, but that just relocates the coupling into a new central coordinator and a

new single point of failure. Event streaming with independent consumers costs more infrastructure and buys into eventual consistency, but it's the only option that fully decouples availability and latency across those six processes.

Why this, and what it costs. Payment latency and availability could not be allowed to depend on fraud scoring, notification delivery, or loyalty crediting, which is why event streaming won out. The cost is real: debugging a distributed flow is harder than reading a call stack, offset here with correlation IDs and distributed tracing (see Observability), and every consumer has to be built idempotent since Kafka delivers at-least-once, not exactly-once.

Database-per-service instead of a shared database

The problem, and what else was on the table. A single shared database is simple to query but creates a shared failure domain and blocks independent deployment — a schema migration in one team's table can lock rows another team depends on. CQRS with a dedicated read model would have offered the best of both, at the cost of a projection pipeline to build and operate; that was set aside in favor of a simpler split for now.

Why this, and what it costs. Fault isolation and independent deployability were non-negotiable given the different regulatory and compliance obligations sitting behind different namespaces — a shared schema would have put unrelated services inside the same audit and blast-radius scope. (This is about compliance scope and audit boundaries, not a claim that namespace separation alone satisfies PCI-DSS — see Known Limitations & Operational Edge Cases.) The tradeoff: no cheap cross-service JOINS, and cross-domain reporting requires purpose-built read models fed by events instead of a live query, with a small, monitored amount of replication lag to account for.

One EKS cluster with seven namespaces instead of one cluster per domain

The problem, and what else was on the table. Namespaces give logical isolation within a shared control plane; separate clusters give a harder boundary but multiply operational overhead — seven sets of upgrades, seven sets of IAM wiring, seven service meshes to keep in sync. A cluster-per-tier middle ground was also evaluated and rejected as adding complexity without a matching isolation benefit.

Why this, and what it costs. At this service count, NetworkPolicy plus Istio's per-namespace mTLS delivers a meaningful operational-isolation benefit without seven times the platform team's workload — a noisy pod in one namespace still can't reach another's database or API over the network. It's worth being precise about what this does and doesn't buy: namespace and mTLS isolation reduce operational blast radius, but they run on a shared kubelet, container runtime, and node OS, so on their own they are not a substitute for the network- and workload-level segmentation a PCI assessor would expect around cardholder-data-adjacent workloads (see Known Limitations & Operational Edge Cases). A dedicated node group with taints for payment-services, stronger runtime isolation, or a fully separate cluster are reasonable future enhancements if that harder boundary is ever required — none of them are implemented today. The tradeoff already accepted is that a catastrophic EKS control-plane failure affects every namespace at once; that's tolerated given AWS's control-plane SLA and multi-AZ design, but would be revisited immediately if a regulator required physical infrastructure separation.

Istio service mesh instead of bare Kubernetes networking

The problem, and what else was on the table. Kubernetes Services and NetworkPolicies alone don't provide encryption in transit between pods, retries, circuit breaking, or fine-grained traffic control. Pushing application-level TLS and retry logic into every codebase was considered and rejected — it just relocates the same problem into a dozen places instead of solving it once.

Why this, and what it costs. Encrypting every pod-to-pod hop and enforcing zero-trust by default was worth the resource cost of an Envoy sidecar per pod, and centralizing retries and circuit breaking meant this logic didn't have to be reimplemented inconsistently across services. The cost: every pod carries sidecar CPU/memory overhead, and the mesh itself becomes a critical piece of infrastructure that has to be upgraded carefully — a bad Istio upgrade is a platform-wide risk, not a single-service one.

OPA for authorization instead of in-service checks

The problem, and what else was on the table. Hard-coded authorization logic scattered across a dozen services means every policy change requires a coordinated multi-service deploy. A centralized authorization service with custom logic was considered but set aside — it would have meant maintaining another bespoke system instead of adopting a standard one.

Why this, and what it costs. OPA with Rego policies decouples policy from code, so compliance-driven access rule changes can ship without touching application code or waiting on a release train. The cost is a real one: an inline dependency added to the request path for any protected resource, and the team has to learn Rego — a small but genuine cost against a more familiar general-purpose language.

Redis for caching and session state instead of database-only reads

The problem, and what else was on the table. Hitting Postgres for every session check or idempotency lookup adds load to the primary database exactly where it can least afford it. In-memory per-pod caching was considered and rejected — it's fast, but inconsistent across replicas and lost on every restart, which doesn't work for a horizontally scaled service.

Why this, and what it costs. Session lookups and idempotency checks needed to be fast and consistent across every pod, and only a shared cache satisfies both. The tradeoff: Redis becomes a dependency whose unavailability degrades — but should not break — multiple services, which is why Auth and Payment Gateway are built to fail safe, falling back to a slower re-validation against Postgres rather than failing the request outright.

PostgreSQL for core financial data instead of a NoSQL store

The problem, and what else was on the table. Financial data needs strong consistency, transactional integrity, and a mature ecosystem for backups, replication, and point-in-time recovery. A distributed SQL database was considered for its consistency at larger scale, but set aside as a newer, less battle-tested operational profile than the team wanted to take on for this workload.

Why this, and what it costs. For ledgers and transaction records specifically, ACID guarantees and mature point-in-time recovery outweighed the horizontal write scalability a NoSQL store would offer — this data isn't write-volume-bound the way NoSQL is optimized for. The accepted limit: vertical scaling on the primary eventually becomes a ceiling, with read replicas absorbing read load while write throughput on Ledger stays bounded by a single primary — acceptable given append-only writes are inherently modest in volume.

Amazon MSK instead of self-hosted Kafka or RabbitMQ

The problem, and what else was on the table. The platform needed a durable, ordered, replayable event backbone that could handle both high-throughput fan-out and long-retention compliance topics. RabbitMQ was considered and rejected for this specifically — it's a queue, not a log, and weaker at the long retention and replay compliance topics like SARFired require.

Why this, and what it costs. Self-hosting Kafka on EC2 or in-cluster would have kept full control, at the cost of the team owning broker upgrades, scaling, and disaster recovery directly; managed MSK removes that operational load without giving up Kafka's durability or replay semantics. The tradeoff: less low-level tuning control than self-hosting, and AWS-specific operational knowledge becomes load-bearing — accepted given the team was already deep in the AWS ecosystem.

EKS instead of ECS

The problem, and what else was on the table. The platform needed a container orchestrator with a mature ecosystem for service mesh, fine-grained autoscaling, and policy enforcement. ECS offered a simpler operational model and tighter native AWS integration, but a noticeably smaller ecosystem for mesh and policy tooling at the time this was decided.

Why this, and what it costs. Istio, OPA, and Falco were core to the security model from day one, and all three are Kubernetes-native — building an equivalent stack on ECS would have meant assembling less mature tooling. The cost accepted: EKS carries meaningfully more operational complexity than ECS, with cluster upgrades, node group management, and CNI configuration as real ongoing costs a smaller team would need to weigh carefully before choosing this path.

Read replicas instead of scaling the primary vertically

The problem, and what else was on the table. Reporting and other read-heavy workloads were competing with live transactional writes for the same database resources. A separate analytics warehouse fed by CDC was considered as the best long-term separation, but set aside as a bigger infrastructure investment than was justified at current scale.

Why this, and what it costs. Decoupling Reporting's bursty analytical queries from Ledger's steady transactional writes was worth a small, monitored amount of replication lag. Reporting and similar consumers see slightly stale data as a result — acceptable because nothing reading from a replica in this platform sits on the customer-facing path.

Failure Scenarios & Resilience

Every dependency in this system fails eventually — a node gets reclaimed, an external API times out, a broker has a bad day. The design goal was never zero failures; it was making sure each failure stays contained to the smallest possible blast radius and recovers without a human being paged at 3 a.m. for most of them.

Scenario	Blast radius	How the platform responds
Payment processor timeout	Single payment request	Bounded retry with exponential backoff and an idempotency key; if retries exhaust, the customer sees a clear failure rather than a hang, and no duplicate charge is possible.
Kafka (MSK) broker unavailable	New events can't be published or consumed	Producers buffer locally and retry against the remaining in-sync replicas (replication factor 3 across AZs); consumers resume from their last committed offset once brokers recover — no events are lost, just delayed.
Redis unavailable	Session lookups and idempotency checks slow down	Auth and Payment Gateway are built to fail safe: they fall back to a slower path (re-validate against Postgres) rather than fail the request outright.
Fraud Detection crashes	Fraud scoring backs up	Payments are unaffected — Fraud consumes asynchronously. Unscored transactions queue in Kafka and are processed once the service recovers; an alert fires on consumer lag exceeding a threshold.
Notification Service crashes	Receipts are delayed	No impact on payment success. Kafka retains the events; the service catches up on restart. A dead-letter topic captures anything that fails repeatedly for manual review.
RDS primary failure	One service's writes are briefly unavailable	Multi-AZ automatic failover promotes the standby, typically inside a minute; the service's connection pool reconnects automatically via the RDS endpoint, which doesn't change.
EC2 node failure	Pods scheduled on that node	Kubernetes reschedules affected pods onto healthy nodes; Pod Disruption Budgets ensure a minimum number of replicas stay available throughout.
Availability Zone failure	Roughly a third of cluster capacity	Multi-AZ node groups plus topology spread constraints mean no service is entirely concentrated in one AZ; Cluster Autoscaler adds capacity in the healthy AZs.
Pod crash / OOMKill	Single pod	Kubernetes restarts it automatically; resource limits are tuned from observed usage so this is a rare event rather than a routine one.

Scenario	Blast radius	How the platform responds
Istio sidecar failure	Single pod's traffic	The pod fails readiness checks and is pulled out of the load-balancing pool until the sidecar recovers, rather than serving traffic without mTLS or policy enforcement.
External payment provider outage	All new payments	Circuit breaker trips after a threshold of failures, returning fast, clear failures instead of letting requests pile up and time out slowly; automatically retries a probe request on an interval to detect recovery.
Spot node interruption	Pods on High-Performance node group	AWS gives a two-minute warning; affected batch and reporting jobs checkpoint or simply restart, since they were selected for this node group precisely because they tolerate interruption.

The mechanisms behind the table

Idempotency: Every write that could be retried carries an idempotency key, so a retried request is detected and returns the original result instead of repeating the side effect.

Circuit breakers: Istio DestinationRule outlier detection trips after a configured error threshold, stopping traffic to a failing upstream instead of letting every caller time out independently.

Dead-letter queues: Events that fail processing after a bounded number of retries move to a dead-letter topic instead of blocking the consumer group's offset indefinitely, so one poison message can't stall an entire pipeline.

The Outbox Pattern: A service writes its state change and the event it needs to publish in the same local database transaction, then a separate publisher reads that outbox table and pushes to Kafka. This closes the gap between "I saved the data" and "I told everyone about it" — the two can never happen only one out of two.

Eventual consistency: Consumers of an event may see it seconds after it happened, not instantly. Every part of this platform that reads across a service boundary is designed around that assumption rather than pretending it isn't there.

Regional Disaster Recovery

Scope of this section

Everything in Failure Scenarios & Resilience above operates within a single AWS region across multiple Availability Zones. This section describes a reference approach for a regional failure — a scenario this platform has not been operationally tested against — rather than reporting it as an implemented, audited capability.

A regulated payments platform is generally expected to have a documented answer for a regional-level event, not just an Availability-Zone-level one — a full region outage is a different failure class with a different blast radius, and multi-AZ resilience alone doesn't cover it. The reference approach below describes a warm standby pattern: the shape a region-level DR plan for this platform would reasonably take, sized to the failure modes already described in this document, without claiming it has been built, drilled, or measured in production.

Topology: A warm standby in a second AWS region, sized to run at reduced capacity — enough to serve payment-services and user-services at a lower throughput ceiling, not a full mirror of primary-region capacity.

Data replication: RDS cross-region read replicas for Postgres, and a Kafka cross-region replication tool (for example MirrorMaker or MSK Replicator) for topics that need to survive a regional loss — most notably compliance-retention topics like SARFiled, where losing history isn't an option.

Promotion: A deliberately manual decision to promote the standby region, given the size of the blast radius and the financial and regulatory consequences of an unnecessary failover. This is not intended to be a fully automatic regional failover.

Illustrative RPO / RTO targets: RPO in the low minutes, bounded by cross-region replication lag on RDS and Kafka; RTO in the range of one to a few hours, bounded by DNS cutover, standby capacity scale-up, and the manual promotion decision. These are reference targets for a design of this shape, not measured figures from a live regional failover.

Explicitly out of scope today: Active-active multi-region traffic serving, automatic regional failover, and a tested/drilled regional runbook are not part of the current architecture. They are the natural next step if regulatory requirements or business risk tolerance demand a harder regional guarantee than warm standby provides.

Kubernetes Operational Model

Kubernetes primitives are chosen deliberately here, not defaulted to. This platform runs no databases inside the cluster — every stateful store is a managed AWS service — so most workloads are simple, stateless Deployments, and the more specialized primitives are reserved for the jobs that actually need them.

Primitive	Used for
Deployment	Every application service — Payment Gateway, Auth, Fraud Detection, and the rest. Stateless by design, so any replica can be killed and replaced without coordination.
StatefulSet	Not used for primary data stores, since Postgres, Redis, and Kafka all run as managed AWS services outside the cluster. Reserved for any future in-cluster components that need stable network identity.
DaemonSet	Falco, which needs exactly one runtime security agent per node to watch syscalls at the host level.
Job / CronJob	Scheduled reconciliation tasks, nightly Reporting exports, and certificate-expiry checks — bounded work that runs to completion rather than serving traffic.
ConfigMap	Non-secret configuration — feature flags, service endpoints, log levels — decoupled from the container image so config changes don't require a rebuild.
Secret + Secrets Store CSI Driver	Mounts AWS Secrets Manager values as files inside pods at runtime, so credentials are never baked into an image or committed to Git.
IRSA (IAM Roles for Service Accounts)	Gives individual pods scoped AWS permissions tied to a Kubernetes service account — no long-lived static AWS keys anywhere in the cluster.
StorageClass / PersistentVolume	Limited use — mainly local buffering for Prometheus and a few batch jobs via the EBS CSI driver, since core data lives outside the cluster.
Service (ClusterIP)	East-west traffic between pods within the mesh.
Ingress / Gateway API	North-south entry via NGINX Ingress Controller today, followed by the Istio Gateway mTLS boundary — two independent L7 proxies in sequence, which costs an extra hop of latency and means routing/TLS config has to be reconciled across two systems during incidents. Gateway API is on the roadmap as the longer-term replacement that collapses this into one hop, but hasn't yet displaced the split NGINX/Istio ingress model in production.
Istio VirtualService / DestinationRule	Traffic splitting for canary releases, retry policy, timeout configuration, and the outlier-detection circuit breaking described above.
NetworkPolicy	Default-deny per namespace, with explicit allow rules for the specific cross-namespace calls each service actually needs.

Scaling & Scheduling

Horizontal Pod Autoscaler: Most services scale on CPU and memory; a few — Fraud Detection, Notifications — scale on Kafka consumer lag instead, since that's the metric that actually reflects their backlog.

Cluster Autoscaler: Adds EC2 nodes when pods are pending and can't be scheduled; Karpenter has been evaluated as a faster, more flexible replacement and is on the roadmap, but hasn't yet displaced Cluster Autoscaler in production.

Pod anti-affinity: Replicas of the same service are spread across different nodes so a single node failure can't take out every replica of anything customer-facing.

Topology spread constraints: Extend that same spreading across Availability Zones, not just nodes, so an AZ failure degrades capacity rather than taking a service fully offline.

Pod Disruption Budgets: Guarantee a minimum number of available replicas during voluntary disruptions — node drains, cluster upgrades, Spot reclamation — so routine maintenance can't accidentally cause an outage.

Priority classes: Payment-services workloads are given higher scheduling priority than other-services, so under resource pressure the cluster protects the money path first.

Resource requests & limits: Set from observed usage rather than guessed — under-provisioned limits cause throttling and OOMKills, over-provisioned requests waste capacity and inflate the node count needed.

Connection pooling (PgBouncer): Sits in front of the busier Postgres instances so a burst of pod replicas doesn't each open its own raw connection and exhaust Postgres's connection ceiling.

Caching strategy: Redis absorbs session lookups, idempotency checks, and Fraud Detection's feature lookups — the three read patterns that would otherwise hit Postgres hardest under load.

Security, End to End

Security here isn't a layer bolted on at the edge — it's enforced at the network, the workload, the data, and the account level simultaneously, on the assumption that any single control will eventually fail or be misconfigured.

Control	Layer	What it actually does
Zero Trust / mTLS (Istio)	Pod-to-pod	Every service call is encrypted and authenticated by default — no pod trusts another just because it's on the same cluster.
OPA	API / policy	Fine-grained, declarative authorization decisions decoupled from application code.
Falco	Node / runtime	Watches syscalls in real time and flags anomalous behavior — a process trying to open a shell in a payment pod, for instance.
AWS WAF & Shield	Edge	Blocks common web exploits and absorbs DDoS traffic before it ever reaches the cluster.
IRSA	Pod-to-AWS	Scoped, short-lived AWS credentials per pod instead of long-lived static keys.
AWS Secrets Manager + CSI Driver	Secrets	Credentials are injected at runtime as mounted files, never stored in a YAML manifest or Git.
Pod Security Standards	Workload	Enforces baseline pod-level restrictions — no privileged containers, no running as root, by default.
Admission control	Cluster	Policy checks run before a workload is ever scheduled, catching misconfiguration at deploy time instead of runtime.
Image scanning (SonarQube / Trivy-class tooling)	CI/CD	Every image is scanned for known vulnerabilities before it's allowed into ECR.

Control	Layer	What it actually does
Network segmentation (NetworkPolicy)	Namespace	Default-deny between namespaces, explicit allow only where a real dependency exists.
CloudTrail / GuardDuty / Security Hub	AWS account	Account-level audit logging, threat detection, and a consolidated view of security findings across the AWS footprint.
cert-manager	TLS lifecycle	Automatic certificate issuance and rotation — no certificate expiry incidents from a forgotten manual renewal.
KMS / envelope encryption	Data at rest	Secrets Manager values and S3 data are encrypted with keys that are themselves encrypted by a master key in KMS, so a leaked data key alone isn't enough to decrypt anything.

Least privilege runs through every one of these: IRSA roles are scoped to exactly what a pod needs, OPA policies default-deny, and NetworkPolicies only open the specific paths a real dependency requires. None of these controls are relied on alone — they're deliberately overlapping, so a gap in one is caught by another.

Network Architecture: Cloud Layer and Kubernetes Layer

The namespace and mesh isolation described earlier operates entirely inside the Kubernetes control plane. Underneath it sits a separate, independent layer of AWS-native network isolation, and the two are complementary rather than substitutes for each other — a gap in one is not automatically closed by the other.

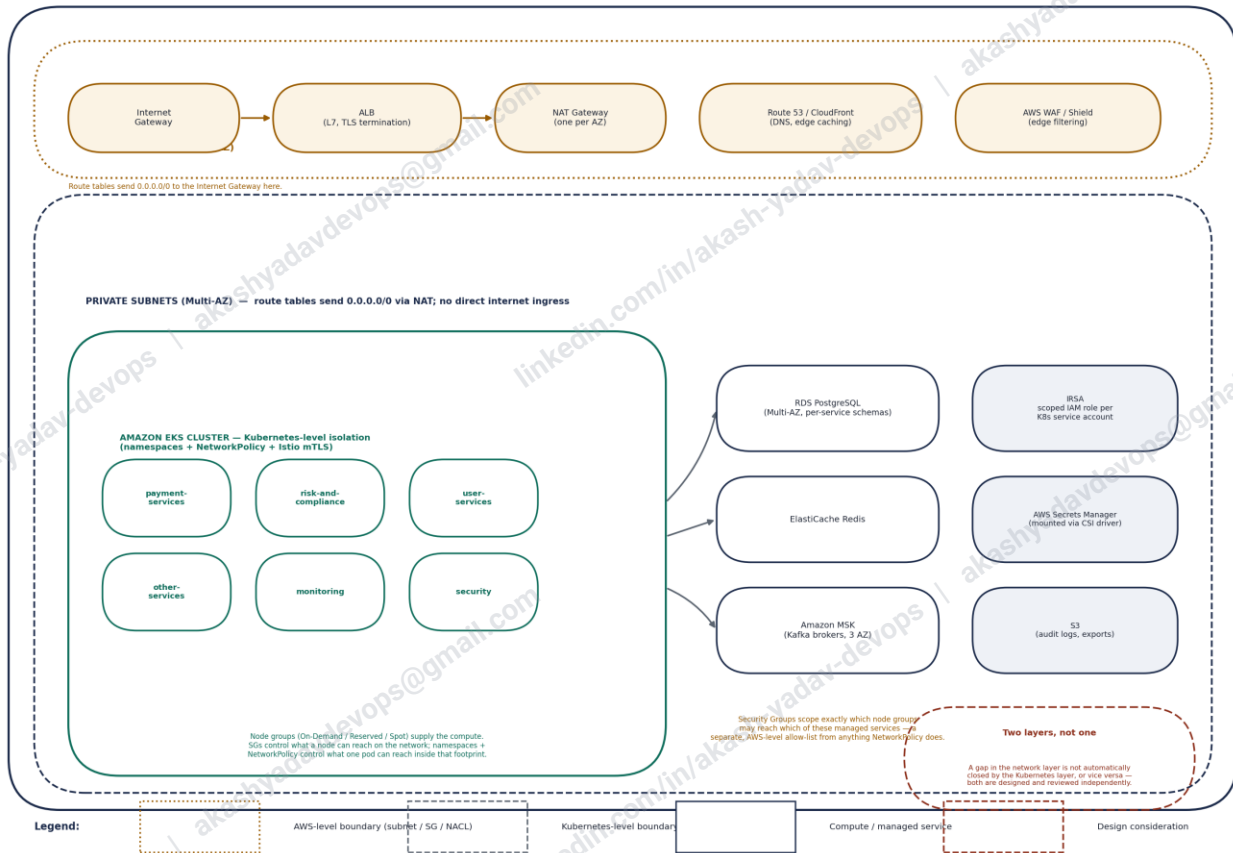
AWS / cloud-level control	What it isolates, independent of Kubernetes
VPC	The account's private network space — nothing outside it can address a resource inside it directly.
Public vs. private subnets	Public subnets hold only edge components (ALB, NAT Gateway); everything else sits in private subnets with no direct route to the internet.
Route tables	Decide whether a subnet's traffic can reach the internet via an Internet Gateway, or only reach out through NAT — this is what actually makes a subnet “private,” not just a label.
Security Groups	Stateful, instance-level allow-lists — which node groups, RDS instances, or MSK brokers may talk to which others, regardless of what's deployed on top of them.
NACLs	Stateless, subnet-level allow/deny rules — a coarser, defense-in-depth layer behind Security Groups.
EKS node groups	The EC2 capacity Kubernetes schedules onto; node groups sit inside private subnets and inherit whatever Security Group is attached to them.
Kubernetes namespaces	A logical boundary enforced by the cluster control plane and CNI, not the VPC — it governs which pods can call which other pods, on top of whatever the network layer already allows.

The distinction matters in practice: Security Groups and NACLs decide what can reach a node or a managed service over the network at all, before Kubernetes ever gets a say. Namespaces, NetworkPolicy, and Istio's mTLS then decide what one pod can reach inside whatever footprint the network layer has already allowed. Neither layer compensates for a misconfiguration in the other, which is why both are designed and reviewed as separate controls rather than treated as one combined guarantee.

Infrastructure & Networking

Two independent isolation layers: AWS network boundaries (VPC / subnets / Security Groups / NACLs) and Kubernetes-level boundaries (namespaces / NetworkPolicy / mTLS).

AWS REGION — VPC (single account)



Cloud-level network boundaries (VPC / subnets / Security Groups / NACLs) shown alongside the Kubernetes-level boundaries (namespaces / NetworkPolicy / mTLS) they sit underneath.

The Request Path

A request entering this platform crosses more hops than it might seem, and each one exists to solve a specific problem rather than to add ceremony:

1. **Client** → 2. CloudFront (edge caching, global latency reduction) → 3. AWS WAF (blocks malicious requests before they cost the platform anything) → 4. ALB (L7 routing, TLS termination) → 5. NGINX Ingress (routes to the right in-cluster service) → 6. Istio Gateway (mTLS boundary into the mesh) → 7. Envoy sidecar (per-pod traffic policy, retries, telemetry) → 8. Service → 9. Pod → 10. Application code.

Pattern	Where enforced	Why it matters here
Load balancing	ALB (north-south), Envoy (east-west)	Spreads traffic evenly and routes around unhealthy pods automatically.
Retries	Envoy / Istio VirtualService	Transient failures (a single dropped connection) are absorbed without surfacing an error to the caller.
Timeouts	Envoy / Istio VirtualService	Bounds how long a caller waits, preventing one slow dependency from cascading into a pile of stuck requests upstream.

Pattern	Where enforced	Why it matters here
Connection pooling	Envoy, PgBouncer	Reuses connections instead of paying setup cost per request, and protects Postgres from connection exhaustion.
Circuit breaking	Istio DestinationRule (outlier detection)	Stops sending traffic to a consistently failing upstream instead of letting every caller time out independently.
Rate limiting	AWS WAF, Envoy	Protects both the edge and internal services from being overwhelmed by a single noisy client or a runaway retry loop.

North-south traffic (client to cluster) and east-west traffic (service to service inside the mesh) are treated as genuinely different problems: north-south is optimized for edge security and global latency, east-west is optimized for encrypted, low-latency, policy-governed calls between trusted workloads.

AWS Networking Architecture: VPC, Subnets, and Traffic Flow

The cloud-level boundaries introduced above sit on top of a fairly standard three-tier VPC design — the layout most production AWS workloads converge on, chosen here for the same reason: it keeps anything that doesn't need to be reachable from the internet from ever being reachable from the internet, independent of anything Kubernetes does.

The VPC is split into three subnet tiers, each duplicated across multiple Availability Zones for redundancy:

Subnet tier	Default route	What lives here	Why
Public subnets	Internet Gateway	Application Load Balancer, NAT Gateways	The only tier with a direct path to the internet — everything here is a controlled entry or exit point, nothing else.
Private application subnets	NAT Gateway (outbound only)	EKS worker nodes across all three node groups, Istio ingress gateway	Where the cluster actually runs. No inbound route from the internet exists; these subnets are only reachable through the ALB in the public tier.
Private database subnets	No route to NAT Gateway or Internet Gateway	RDS PostgreSQL (Primary + Replica), ElastiCache Redis, Amazon MSK brokers	Fully isolated from the internet in both directions — the only traffic these subnets carry originates from the application subnets, over specific ports, from specific security groups.

An Internet Gateway attached to the VPC gives the public subnets their only path in or out to the internet. Every other subnet reaches the internet, if at all, through a NAT Gateway sitting in the public tier, which lets outbound calls out — image pulls from ECR, calls to the payment processor, Secrets Manager, MSK's TLS endpoints — without ever accepting an inbound connection initiated from outside the VPC. Each subnet tier has its own route table: the public route table points 0.0.0.0/0 at the Internet Gateway, the private application route table points 0.0.0.0/0 at the NAT Gateway, and the private database route table has no default route to the internet at all — it only knows about traffic inside the VPC.

Security Groups do the fine-grained work at the resource level — the RDS security group, for example, only accepts inbound traffic on 5432 from the EKS node security group, nothing else, from nowhere else. Network ACLs sit at the subnet boundary as a coarser, stateless backstop: they're the layer that would still block a misconfigured security group from letting an entire subnet talk to something it shouldn't. Both are enforced by AWS, entirely independent of anything running inside the cluster.

VPC Endpoints for S3 and ECR let workloads in the private application subnets reach those AWS services without the traffic ever leaving the AWS network or passing through the NAT Gateway — less NAT Gateway data-processing cost, and one less path that has to be reasoned about in a security review.

End-to-end, tying the AWS layer to the Kubernetes layer described earlier: a client request reaches the Application Load Balancer in the public subnets — the only load balancer with a public IP anywhere in this design. The ALB forwards to the NGINX Ingress Controller and Istio Ingress Gateway running on EKS nodes in the private application subnets, and the request has now crossed from AWS-managed networking into pod networking, where Istio's mTLS and NetworkPolicy take over enforcement. Traffic to a database follows the same principle in reverse: a pod in the private application subnets is the only thing with a network path to the private database subnets at all, and even then, only over the port and protocol its security group allows. AWS networking decides what can reach a subnet or a managed service; Kubernetes networking decides what can reach a namespace, a pod, or an API once a request has already arrived inside the cluster. Neither layer substitutes for the other, and a gap in one isn't automatically closed by the other — the same principle called out earlier for namespaces and Security Groups holds at every layer of this design.

Observability

The goal isn't collecting metrics for their own sake — it's being able to answer "what's broken and why" in minutes during an incident, not hours.

RED metrics (per service): Rate, Errors, Duration — the first thing checked for any individual service during an incident.

USE metrics (per resource): Utilization, Saturation, Errors — used for infrastructure (nodes, databases, brokers) rather than application-level services.

Golden Signals: Latency, traffic, errors, and saturation, tracked together on Grafana dashboards as the top-level view of platform health.

Correlation IDs: A single ID is generated at the edge and propagated through HTTP headers and Kafka message headers, so a single payment can be traced across every synchronous and asynchronous hop it touches.

Distributed tracing (Jaeger + OpenTelemetry): Every hop — including the async ones through Kafka — is stitched into one trace, which is what makes debugging a six-service fan-out from a single payment event tractable.

SLOs, SLIs, and error budgets: Each critical service (Auth, Payment Gateway, Transaction Processor) has a defined SLO with a measurable SLI behind it; the error budget determines how much risk the team can spend on faster releases before shifting focus to stability.

Alertmanager: Routes alerts to PagerDuty, Slack, and on-call rosters based on severity, with every alert linked to a runbook rather than left for the on-call engineer to improvise a response.

Database Design

Primary + read replica: Each service's Postgres instance runs Multi-AZ with a synchronous standby for failover and a separate read replica for read-heavy consumers like Reporting.

Backups & PITR: Automated daily snapshots plus continuous transaction log shipping give point-in-time recovery down to a specific second, not just the last nightly backup.

Connection pooling: PgBouncer sits in front of the busiest instances so pod autoscaling doesn't translate directly into a connection storm against Postgres.

Indexing & partitioning: High-volume tables like Ledger entries are partitioned by month, keeping individual partitions small enough that indexes stay fast and old partitions can be archived independently.

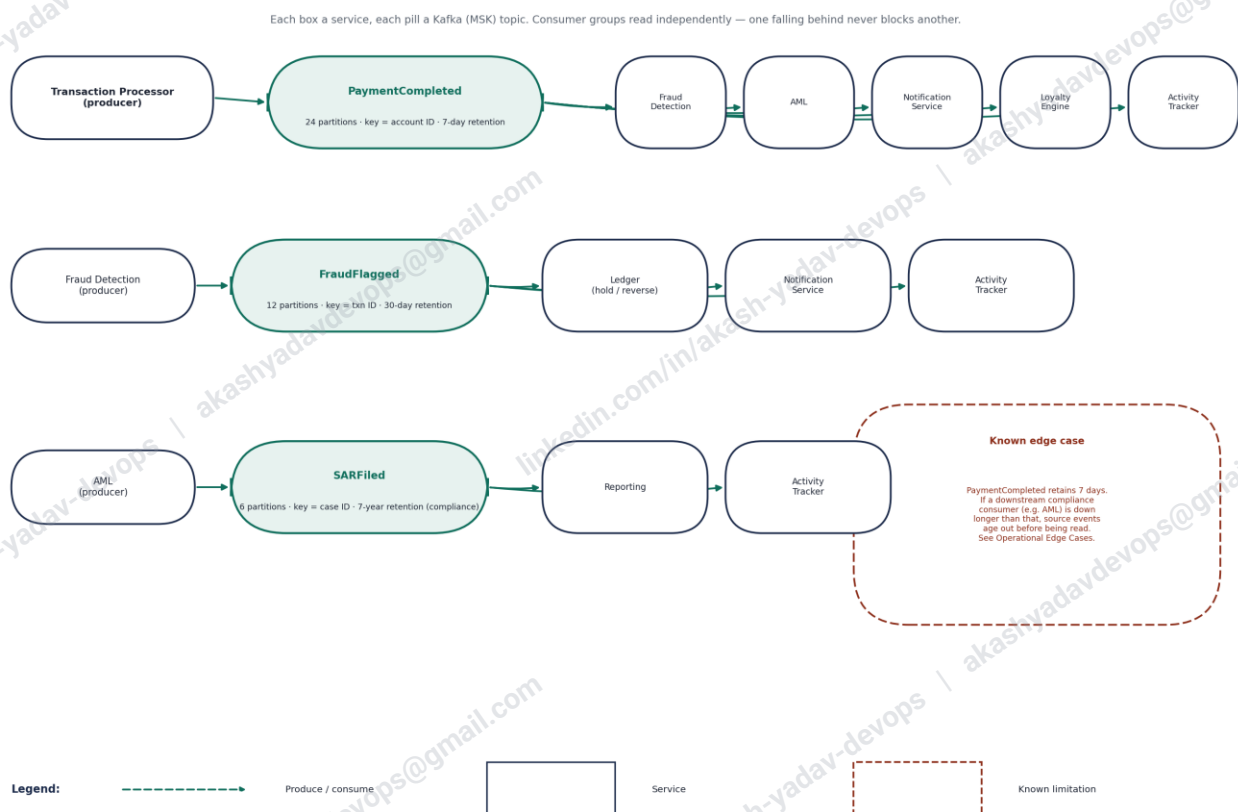
VACUUM & autovacuum tuning: Append-heavy, insert-only tables like the ledger are tuned for the write pattern they actually see, rather than left on Postgres defaults built for a general mixed workload.

Archiving: Cold data beyond the active retention window moves to S3, keeping the operational database lean without discarding the compliance record.

Consistency model: Strong consistency within a single service's own database; eventual consistency across service boundaries, mediated entirely through Kafka events rather than any cross-service query.

Kafka in Depth

Event-Driven Architecture — Kafka Topics & Consumers



Event-driven architecture: producers, topics, and consumer groups across the payment fan-out. The 7-day / 7-year retention gap called out here is discussed under Known Limitations & Operational Edge Cases.

Topics & partitions: Each event type gets its own topic, partitioned by a key chosen for the access pattern — PaymentCompleted is keyed by account ID, which keeps all of one customer's events in order on the same partition.

Replication: Replication factor 3 across Availability Zones, so a single broker or AZ loss doesn't lose data or stop the topic from being readable.

Consumer groups: Each downstream service reads with its own consumer group, so every consumer sees every event independently — Fraud falling behind never affects Notifications' offset.

Ordering: Guaranteed only within a partition, which is why partition keys are chosen deliberately — anything that needs strict per-entity ordering is keyed on that entity's ID.

Retention: Operational topics retain 7–30 days; compliance-relevant topics like SARFiled are retained for years to match regulatory requirements, with older data exported to S3.

Retry topics & dead-letter queues: A failed message gets a bounded number of retries on a dedicated retry topic before landing on a dead-letter topic for manual investigation, so it can never silently block the rest of a partition.

Schema registry: Event contracts are versioned centrally, so a producer can evolve a schema (adding a field) without breaking consumers that haven't updated yet.

Idempotent consumers: Every consumer tracks processed event IDs and treats a duplicate delivery as a no-op — required because Kafka's delivery guarantee here is at-least-once, not exactly-once.

At-least-once over exactly-once: Exactly-once semantics add real operational complexity across a Kafka deployment this size. At-least-once delivery plus idempotent consumers achieves the same practical outcome — no duplicate side effects — with a simpler, more debuggable system.

Known Limitations & Operational Edge Cases

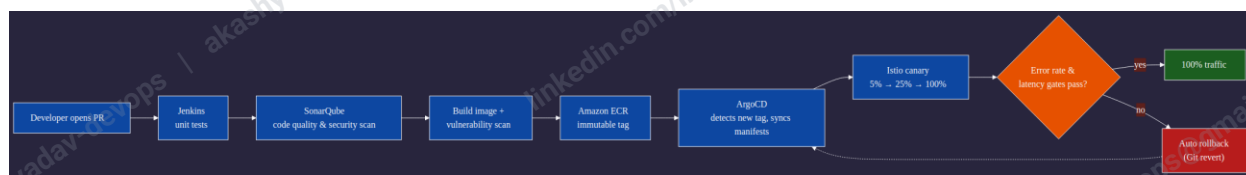
Compliance event retention vs. consumer downtime: PaymentCompleted retains 7 days. If a downstream compliance consumer such as AML were down longer than that, the source events for that gap would age out of the topic before being read, even though SARFiled itself retains 7 years once a case is actually filed. There is no compensating longer-retention mirror of PaymentCompleted today — extending retention on that topic, or archiving it to S3 the way compliance topics already are, is the natural fix if this risk isn't acceptable as-is.

Single-region scope: Every resilience mechanism described in Failure Scenarios & Resilience operates within one AWS region across multiple Availability Zones. See Regional Disaster Recovery for what a region-level failure would mean and the reference approach for handling it — that is a documented design, not something currently built and drilled.

Namespace isolation is not a compliance scope reduction on its own: See Architecture Tradeoffs — One EKS cluster with seven namespaces. Namespace and mesh isolation reduce operational blast radius; they do not, by themselves, satisfy what a PCI assessor would expect for cardholder-data segmentation without additional controls.

Reduced exposure, not a compliance verdict: Payment Gateway forwards card data directly to the processor rather than tokenizing it in-house (see Payment Gateway, under The Services, One at a Time). Because cardholder data never lands in this platform's own databases or logs, routing it directly to a PCI-DSS-compliant processor significantly reduces the amount of sensitive payment data this platform itself handles and stores. The exact PCI-DSS scope any deployment carries, though, depends on the complete implementation and the organization's surrounding controls — network segmentation, access management, logging, and more — not on this architecture pattern alone. Namespace-level isolation (see Architecture Tradeoffs above) is a reasonable technical boundary, but it is not a compliance determination: the platform's actual PCI-DSS scope must always be validated through a formal assessment by a Qualified Security Assessor (QSA), never inferred from this document.

CI/CD Pipeline



Standalone CI/CD & GitOps flow — PR to production, with the canary gate and rollback path called out separately from the request-handling diagrams above.

1. **Developer opens a PR** → 2. Jenkins runs unit tests → 3. SonarQube scans for code quality and security issues → 4. Docker image is built and scanned for vulnerabilities → 5. Image is pushed to Amazon ECR with an immutable tag → 6. ArgoCD detects the new tag and syncs the Kubernetes manifests → 7. A canary rollout shifts a small slice of traffic → 8. Automated metrics gates check error rate and latency → 9. Traffic ramps to 100% or the rollout is automatically rolled back.

Canary releases: Istio traffic splitting shifts a small percentage of live traffic (typically 5%, then 25%, then 100%) to the new version, with error rate and latency gates between each step — a regression is caught against real traffic before it reaches everyone.

Rollback: A rollback is a Git revert, not a manual kubectl command — ArgoCD detects the reverted state and reconciles the cluster back automatically, which removes human error from the riskiest moment in a deploy.

Blue-green, considered and set aside for payment-services: Blue-green gives an instant full-traffic cutover and an equally instant rollback, but it doubles infrastructure cost for the duration of a release. Canary was chosen instead because it catches regressions on a small slice of real traffic before they're customer-wide, which matters more here than an instant cutover.

GitOps: Git is the single source of truth for what should be running in the cluster. ArgoCD continuously reconciles actual cluster state against the Git repository — nobody runs kubectl apply against production by hand.

Production Metrics

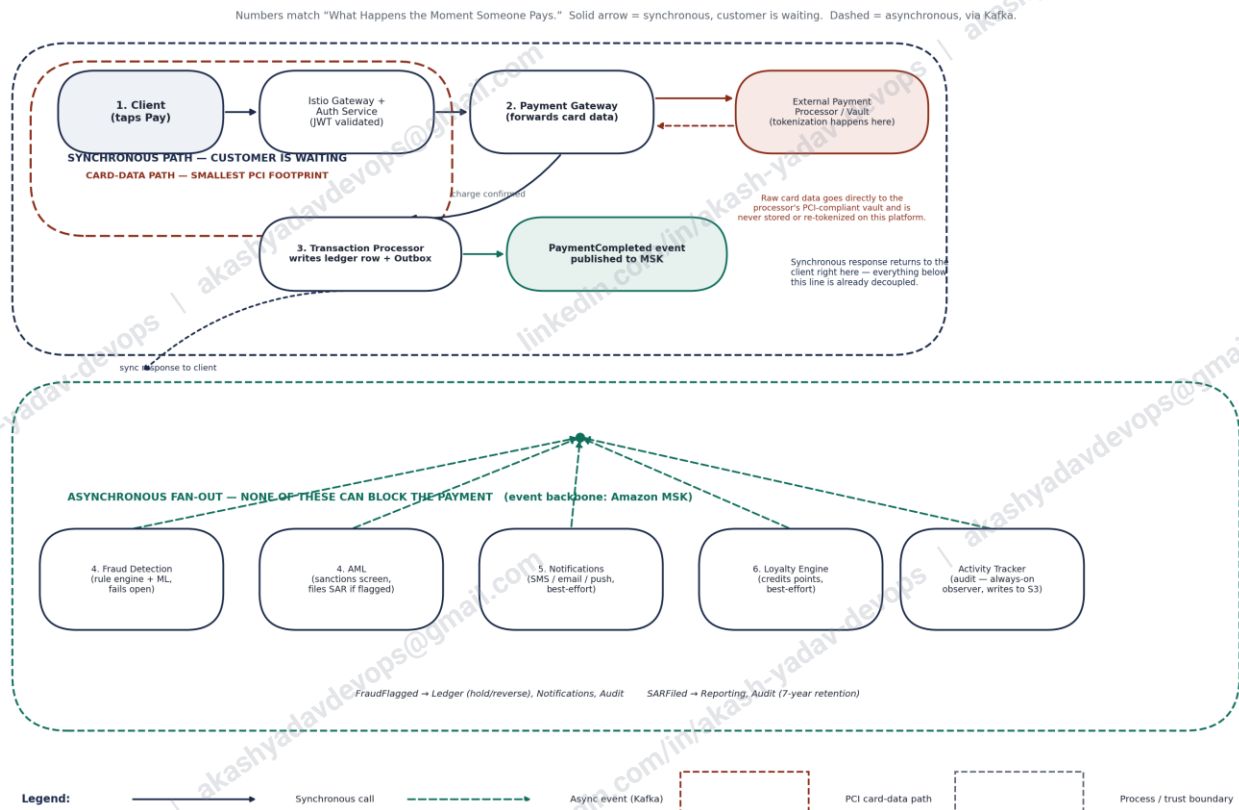
Illustrative figures The numbers below are representative of a platform operating at this scale and are shown to give a sense of order of magnitude. They are not disclosed production measurements.	
Metric	Illustrative value
Peak requests/sec	~4,000–6,000 across the platform
Transactions/day	~1.2M
Kafka messages/day	~15M across all topics
Payment P95 latency	~280ms
Payment P99 latency	~650ms
Monthly availability	99.95% for payment-services
Error rate (payment path)	<0.05%
Deployment frequency	Multiple times per day (other-services), weekly (payment-services)
Mean time to recovery	<15 minutes for automated failovers, <1 hour for human-in-the-loop incidents
Peak node count	~90–110 EC2 instances across three node groups

What Made This Work Under Real Production Pressure

Three decisions carried more weight than any others, and each is covered in more depth in Architecture Tradeoffs above: no shared databases anywhere, Kafka as the backbone instead of chained REST calls, and security enforced at every layer rather than bolted on at the edge. None of these came from a best-practices checklist — each reflects lessons commonly learned in production environments where earlier designs exposed limitations around shared databases, tightly coupled synchronous calls, or edge-only security, and were rebuilt around those failure modes.

What Happens the Moment Someone Pays

Payment Request Lifecycle



Payment Request Lifecycle — synchronous path, PCI card-data path, and the asynchronous fan-out, numbered to match the walkthrough below.

- 1. Request arrives** — ALB routes the request through the Istio gateway. The JWT is validated by Auth Service before the request is allowed anywhere near Payment Gateway.
- 2. Payment is processed** — Payment Gateway charges the external processor. Transaction Processor records the ledger entry and publishes PaymentCompleted to MSK via the Outbox Pattern.
- 3. Fraud check runs, async** — Fraud Detection consumes the event, runs the rule engine and ML model, and flags or clears the transaction independently of everything else happening.
- 4. AML check runs, async** — AML consumes the same event, checks it against compliance rules, and files a SAR if flagged — otherwise clears it. (KYC is a one-time onboarding check, not a per-payment step — see KYC Verification under The Services, One at a Time.)
- 5. Notifications go out** — Notification Service consumes the event and sends the SMS receipt, email confirmation, and push alert at the same time.
- 6. Audit and analytics close it out** — Notifications, audit, and downstream analytics complete the payment lifecycle. Notification Service delivers customer confirmations while Activity Tracker records an immutable audit trail. Reporting and analytics consume the same Kafka event asynchronously, ensuring operational visibility without adding latency to the payment path.

Loyalty Engine, along with the rest of the other-services namespace, also receives PaymentCompleted asynchronously — it's an optional downstream business consumer of the payment event, not a step on the critical payment path (see Loyalty Engine under The Services, One at a Time).

Full Technology Reference

Layer	Technology
Container Orchestration	Kubernetes — Amazon EKS, multi-AZ node groups
Service Mesh	Istio — mTLS enforcement, traffic policies, circuit breaking
CI/CD & GitOps	Jenkins, SonarQube, Amazon ECR, ArgoCD
Infrastructure as Code	Terraform — all AWS resources, EKS cluster, networking
Event Streaming	Amazon MSK (Managed Kafka) — async inter-service backbone
Databases	RDS PostgreSQL (Primary + Read Replica), ElastiCache Redis
Object Storage	Amazon S3 — documents, audit logs, compliance exports
Observability	Prometheus, Alertmanager, Grafana, Jaeger, OpenTelemetry
Security	Falco, OPA, AWS Secrets Manager, Secrets Store CSI Driver, cert-manager
DNS & CDN	Route 53, CloudFront, ExternalDNS
Edge Security	AWS WAF, AWS Shield, NGINX Ingress Controller

© Akash Yadav · Reference architecture for educational and portfolio purposes